

PrivacyGNN

A learner's guide to the whole project
From research goals to code and outputs

Krish Sharma (with PI Dr. Shaikh Arifuzzaman)

May 10, 2026

Abstract

This guide explains the **PrivacyGNN** codebase in plain language: what problem it studies, how the pieces connect, and where to look in the repository. It moves from the big picture (privacy and graph learning) down to files, configuration, metrics, and known limitations of this checkout. Read it once top-to-bottom, then use the file index as a map when you dive into code.

Contents

1	The big picture: what this project is	2
2	Why graphs make privacy interesting	2
3	Research questions (what the experiments answer)	2
4	Repository map (what lives where)	3
5	Configuration-driven experiments	3
6	Data: real graphs and synthetic controls	4
7	Models and training	4
8	Membership inference attacks (how leakage is measured)	4
9	One experiment end-to-end (run_one)	5
10	Running the project and outputs	5
11	How to read the results columns	6
12	Mental model: experiment grid size	6
13	Known limitations in this checkout (honest map)	6
14	Suggested learning path for you	6

1 The big picture: what this project is

PrivacyGNN is a research codebase that asks: *Do graph neural networks (GNNs) leak information about which data points were used for training?* That question is formalized with **membership inference attacks** (MIAs): algorithms that try to guess whether a given example was in the training set, using only the trained model’s predictions (or related signals).

The project compares:

- **Graph models** (GCN, GraphSAGE) that use both node features and edges, versus
- **Non-graph baselines** (logistic regression, MLP) that use features only.

It also tests whether simple **defenses** (DropEdge, label smoothing, early stopping, confidence masking, edge sparsification, and an optional DP-SGD-style path) reduce attack success without destroying task accuracy.

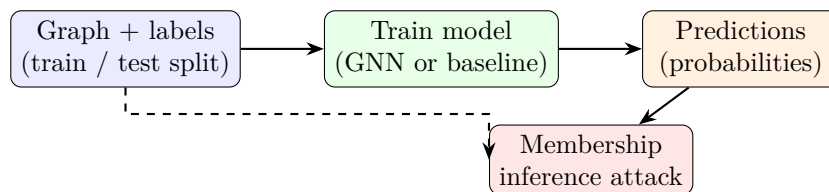


Figure: Training produces a model; attacks use outputs (and implicitly know train vs. test membership for evaluation). The dashed line reminds you that the attacker in the real world does not see ground-truth membership—we only use it to score the attack.

2 Why graphs make privacy interesting

On images or tabular rows, each training example is often independent. On a **graph**, nodes are linked: a node’s prediction depends on its neighbors. So “membership” is entangled with structure—a node’s training signal propagates through edges. This project measures leakage in that setting and relates it to measurable graph statistics:

- **Homophily**: fraction of edges connecting same-label nodes (high = neighbors tend to agree).
- **Density**: how many edges exist relative to all possible edges.

Synthetic datasets in `data.py` vary homophily (**high** / **low**) and density (**sparse** / **medium** / **dense**) *orthogonally*, so you can study structure effects in a controlled way. Citation networks (Cora, Citeseer) provide realistic benchmarks.

3 Research questions (what the experiments answer)

1. **RQ1**: Are GNNs more vulnerable to MIAs than non-graph baselines (MLP / logistic regression)?
2. **RQ2**: How do homophily and density correlate with leakage?
3. **RQ3**: Do lightweight defenses reduce leakage with acceptable utility loss?
4. **RQ4**: What does the utility–privacy tradeoff look like?

The pipeline produces accuracy, F1, AUROC (task performance), several attack AUC/accuracy scores, and calibration error (ECE) so you can relate *utility* and *privacy* numerically.

4 Repository map (what lives where)

The active code lives under `privacy-gnn/`. Think of it in layers:

Entry: `run_final.py` — loads config, loops all experiments, writes CSVs and stats.

One run: `experiment.py` — `run_one`: load data, train target, train shadow models if needed, run attacks, return one metrics dict.

Building blocks: `data.py` (datasets), `models.py` (GCN/SAGE), `training.py` (full-batch training + defenses), `attacks.py` (MIA + ECE).

Config: `experiment_config.yaml` + `config.py` — grid of datasets $\times$ models $\times$ defenses $\times$ seeds.

Analysis: `stats_utils.py` (paired *t*-tests, bootstrap CIs), `generate_figures.py` (plots from `all_results.csv`).

Paper assets: `paper/ieee_privacy_gnn.tex`, `paper/build_manuscript.py`, and related notes (`JOURNAL_REPORT.md`, `literature_review.md`).

5 Configuration-driven experiments

You rarely need to edit Python to change the experimental grid. `config.py` reads `experiment_config.yaml` (or the path in environment variable `PRIVACYGNN_CONFIG`).

Key YAML sections:

- **seeds** — list of random seeds (each full combination runs once per seed).
- **datasets** — e.g. Cora, Citeseer, `synthetic_high_sparse`, ...
- **models** — LogReg, MLP, GCN, GraphSAGE.
- **defenses** — list of name + params; `none` is the baseline.
- **attacks** — which MIAs to run: `confidence`, `threshold`, `shadow`, `lira`.
- **training** — epochs, learning rate, weight decay passed into `run_one`.
- **bootstrap**, `lira`, `minibatch`, `dp_sgd` — advanced options (see below).

Rules enforced in code (`get_experiment_list` in `config.py`):

- LogReg and MLP only run with defense `none` (graph-specific defenses apply to GNNs).
- Defense `dp_sgd` only applies to GCN and GraphSAGE, and optionally only on datasets listed in `dp_sgd_datasets`.

For a smaller, paper-aligned grid, use `experiment_config_paper.yaml`:

```
PRIVACYGNN_CONFIG=experiment_config_paper.yaml python run_final.py
```

6 Data: real graphs and synthetic controls

Cora / Citeseer (`load_citation`): loaded via PyTorch Geometric Planetoid; a mirror URL is tried if the default download fails.

Synthetic graphs (`make_synthetic`): fixed number of nodes and features; labels drawn from class centers; edges sampled to hit a target density, with a controllable fraction of same-label vs. different-label edges (homophily). Train/test is a 50/50 split (with optional `resplit` for new splits per seed).

Large-graph path (`ogb_loader.py`, `graph_minibatch.py`): in *this repository snapshot*, `MINIBATCH_DATASETS` is empty and minibatch/DP functions raise `NotImplementedError`. The YAML may mention `ogbn-arxiv` or `Reddit` as future extensions; the learning guide reflects the code as checked in.

7 Models and training

GNNs (`models.py`): two-layer GCN and GraphSAGE (64 hidden units), dropout between layers.

Training (`training.py`, full-batch): Adam, cross-entropy on masked training nodes. Optional:

- **DropEdge**: randomly drop edges each epoch during training.
- **Label smoothing**: replace one-hot targets with softened distributions.
- **Early stopping**: track training loss improvement; restore best weights.
- **Edge sparsification**: remove a fraction of edges once before training.

Baselines: logistic regression and sklearn MLP fit on node features only (no graph), in `experiment.py`.

Confidence masking is applied to *predicted probabilities* after training: keep only top- k class masses and renormalize, to obscure fine-grained confidence.

8 Membership inference attacks (how leakage is measured)

All attacks here are **evaluated** using the known train/test split: training nodes are labeled “member” and test nodes “non-member.” The attacker sees model outputs and tries to classify membership. Attack AUC = 0.5 means no better than random; higher AUC suggests more leakage.

Feature vector (four dimensions, `extract_features` in `attacks.py`) per node:

1. Maximum predicted probability.
2. Predicted probability of the *true* label.
3. Entropy of the predictive distribution.
4. A confidence margin term derived from max probability.

Confidence attack: train a logistic regression to distinguish members vs. non-members using these features; report AUC and accuracy on a held-out half of the combined set.

Threshold attack: uses true-label confidence only, with a median threshold (also returns AUC/acc).

Shadow-model attack: train the same type of model on a *shadow* dataset (same family as the target, different random split/seed), build features from shadow member/non-member

predictions, train an attack model, then evaluate on the *target* model’s members and non-members.

LiRA slot: config may include `lira`. In this checkout, `lira_attack.py` returns chance-level 0.5 / 0.5 as a placeholder when a full LiRA implementation is not present.

ECE: expected calibration error on test predictions (`calibration_error`)—not an attack, but relates to how “trustable” confidences are.

9 One experiment end-to-end (`run_one`)

1. Load dataset and compute homophily and density statistics.
2. Set random seeds (NumPy, PyTorch).
3. Build defense-specific training kwargs (DropEdge rate, etc.).
4. Train the **target** model; obtain full-graph probabilities (or baseline `predict_proba`).
5. Apply confidence masking to probabilities if configured.
6. Compute task metrics on the test mask: accuracy, macro F1, multiclass AUROC (with fallback).
7. If enabled, run confidence/threshold attacks comparing train vs. test outputs.
8. If shadow/LiRA need shadows, train one or more shadow models with independent seeds/splits, collect their probabilities, then run shadow attack and/or LiRA placeholder.
9. Compute ECE on test predictions.
10. Return a flat dictionary of metrics (one row for CSV).

10 Running the project and outputs

Setup:

```
cd privacy-gnn
pip install -r requirements.txt
python run_final.py
```

Main outputs (under `results/` unless overridden):

- `all_results.csv` — one row per (dataset, model, defense, seed).
- `summary.csv` — aggregated mean/std by (dataset, model, defense).
- `significance.csv` — exploratory paired *t*-tests: each defense vs. `none` on attack AUC (`conf_attack_auc`), matched by seed.
- `significance_lira.csv` — same for LiRA AUC if column exists.
- `summary_bootstrap.csv` — bootstrap confidence intervals over seeds (not *p*-values).
- `all_results.partial.csv` — periodic checkpoint during long runs.

Figures: `python generate_figures.py` reads `all_results.csv` and writes PNGs to `figures/`.

Manuscript PDF: `python paper/build_manuscript.py` (see `paper/`).

Column	Meaning
homophily, density	Graph statistics for that run.
test_accuracy, test_f1, test_auroc	Task utility on test nodes.
conf_attack_auc, conf_attack_acc	Classifier-based MIA.
threshold_attack_auc, threshold_attack_acc	Threshold on true-label confidence.
shadow_attack_auc, shadow_attack_acc	Shadow-model MIA.
lira_attack_*	LiRA metrics (placeholder ≈ 0.5 in this checkout).
ece_test	Calibration error (lower is better calibrated).
dp_epsilon	Privacy budget if <code>dp_sgd</code> ran (NaN otherwise).

11 How to read the results columns

12 Mental model: experiment grid size

The total number of runs is approximately

$$N_{\text{runs}} \approx (\#\text{datasets}) \times (\#\text{models}) \times (\#\text{valid defenses}) \times (\#\text{seeds}),$$

where “valid defenses” excludes combinations the code skips (for example, graph-only defenses on LogReg/MLP). The default YAML uses many seeds and defenses, so the full grid can be hundreds of runs—plan runtime accordingly or trim the YAML for development.

13 Known limitations in this checkout (honest map)

- **Large-graph / OGB / Reddit:** not wired in this snapshot; stubs raise errors if selected.
- **LiRA:** placeholder returns chance-level scores; interpret `lira_*` accordingly.
- **Statistical tests:** paired *t*-tests are *exploratory*; bootstrap CIs quantify uncertainty over seeds but are not hypothesis tests.
- **DP-SGD:** would require the minibatch DP path in `graph_minibatch.py` in a full implementation.

14 Suggested learning path for you

1. Read this PDF once (general $\rightarrow$ specific).
2. Open `experiment_config.yaml` and trace one triple (dataset, model, defense) you care about.
3. Read `run_final.py` (loop) then `experiment.run_one` (single run).
4. Read `attacks.py` to internalize the four-dimensional feature map.
5. Skim `data.make_synthetic` to see how homophily and density are enforced.
6. Run a *tiny* grid: one synthetic dataset, two models, two defenses, two seeds—inspect `all_results.csv`.
7. Run `generate_figures.py` and match each figure to filters in the plotting code.

End of guide. For publication narrative and citations, align with `paper/ieee_privacy_gnn.tex` and your advisor’s framing of contributions.